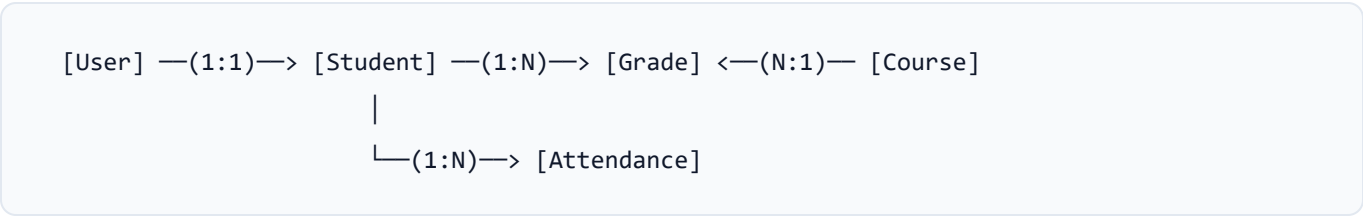


Database Schema Planning - Student Performance Analytics Portal

This document outlines the MongoDB schema designs, validation rules, indices, and relationships. Mongoose is utilized as the ODM to enforce schemas at the application layer.

Entity-Relationship Mapping

MongoDB is non-relational, but schemas reference other collections via `mongoose.Schema.Types.ObjectId` (represented as standard 12-byte Hex IDs).



Collections Details

1. Users Collection

Stores the authentication details and general access profiles of all system users (Admins, Teachers, Students).

Field	Type	Validation	Description
<code>_id</code>	ObjectId	Primary Key	Unique MongoDB document identifier
<code>name</code>	String	Required	Full display name of the user

Field	Type	Validation	Description
email	String	Required, Unique, Email Match	Login email address
password	String	Required, minlength: 6, select: false	Hashed password string
role	String	Required, enum: admin , teacher , student	Authorization level of user
createdAt	Date	Auto	Timestamp of record creation
updatedAt	Date	Auto	Timestamp of last record update

2. Students Collection

Holds specific registration and academic demographic information for students. Refers directly back to the `Users` collection.

Field	Type	Validation	Description
_id	ObjectId	Primary Key	Unique document identifier
userId	ObjectId	Required, Ref: User	Maps student details to their base User login account
studentId	String	Required, Unique	Academic registration number (e.g. STU2026001)
class	String	Required	Grade/Class room designation (e.g. 10-A , 12-C)
enrollmentDate	Date	Default: Now	Date the student enrolled in the institution
guardianName	String	Required	Parent/Guardian contact name
guardianPhone	String	Required	Parent/Guardian phone number

Index: Single index on `class: 1` to speed up teacher analytics queries when filtering performance by class.

3. Courses Collection

Contains the metadata of courses taught at the institution.

Field	Type	Validation	Description
<code>_id</code>	ObjectId	Primary Key	Unique document identifier
<code>code</code>	String	Required, Unique	Unique subject/course code (e.g. <code>CS101</code> , <code>MATH202</code>)
<code>name</code>	String	Required	Display name of the course
<code>teacherId</code>	ObjectId	Required, Ref: <code>User</code>	Reference to the Teacher User managing this course

4. Grades Collection

Stores exam and assignment mark details. Refers to both the Student and the Course documents.

Field	Type	Validation	Description
<code>_id</code>	ObjectId	Primary Key	Unique document identifier
<code>studentId</code>	ObjectId	Required, Ref: <code>Student</code>	Reference to the Student receiving this grade
<code>subjectId</code>	ObjectId	Required, Ref: <code>Course</code>	Reference to the Course the grade applies to
<code>marksObtained</code>	Number	Required, min: 0	Number of marks awarded
<code>maxMarks</code>	Number	Required, default: 100	Maximum possible score for this assessment
<code>examType</code>	String	Required, enum: <code>Quiz</code> , <code>Assignment</code> , <code>Midterm</code> , <code>Final Exam</code> , <code>Project</code>	Assessment categorization
<code>semester</code>	String	Required	Semester/Term (e.g., <code>Fall 2026</code> , <code>Spring 2026</code>)
<code>date</code>	Date	Default: Now	Date of assessment logging

5. Attendance Collection

Tracks daily student attendance records.

Field	Type	Validation	Description
<code>_id</code>	ObjectId	Primary Key	Unique document identifier
<code>studentId</code>	ObjectId	Required, Ref: <code>Student</code>	Reference to the Student
<code>date</code>	Date	Required	Calendar date of the log
<code>status</code>	String	Required, enum: <code>Present</code> , <code>Absent</code> , <code>Late</code>	Student presence classification
<code>remarks</code>	String	Optional	Notes or reason (e.g., "Sick leave with note")

Index: Compound index on { `studentId: 1`, `date: 1` } (unique constraint) to enforce a single attendance state per student per day.